Save Your Save: Allocating Save-Scumming Attempts Under Limited Playtime

COMP1110 Computing and Data Science in Everyday Life · 2026/27 Semester 1

Group Project Plan · Hybrid project (algorithmic + simulation/modelling)

Name	UID	Lead
Vu Q. Nguyen *(Daniel)*	3036755728	Dynamic program, policy map, and the uncertain-odds module
Yerassyl Taisarinov *(Yera)*	3036459776	Monte Carlo simulator, figures, and the experiment pipeline
Jihun Choi *(Roy)*	3036734023	Experiment design, comparative analysis, and the Pokémon case study
Yusif Seyidov *(Yusif)*	3036538850	Team coordination, ethics, and report quality
Vivaan Kandhari *(Vivaan)*	3036684400	Literature, research gap, ρ-baseline, and the Path of Exile snapshot

1. Problem statement

Research question. In a sequence of random game events that share one finite real-time budget, and whose odds may be known or uncertain, when is retrying after a bad outcome worth the time it takes away from later, possibly more valuable, events?

1.1 Context

Save-scumming is the practice of saving before a random in-game event (a dialogue check, a loot drop, a crafting roll, or a legendary encounter) and reloading until a desirable outcome occurs. The same choice appears whenever a player can repeat an attempt for a better result, whether by reloading a save or by farming a boss for a rare drop. We therefore use "reload" and "retry" interchangeably.

The decision is not a single expected-value calculation. For an isolated event, one probability and one reward are enough. With several events in sequence, each retry consumes time that could otherwise be spent on later events. Repeatedly chasing a 5% rare roll can be reasonable when little of value remains, and a poor use of the same minutes when a high-value event is still ahead. The same unsuccessful roll can therefore call for different actions at different points in a session.

Players rarely announce a budget, but deadlines impose one: an exam the next morning, an expiring subscription, a time-limited event, or the end of an online season. Events not reached before the deadline are missed. That *limited-playtime* setting is our main problem. As a baseline we also study *unlimited playtime*, where no event can be missed but time still has an outside value, so the player maximises reward per second (Section 2.3). Comparing the two shows how much a deadline changes the best accept-or-retry rule.

1.2 Scope

In scope: independent, repeatable events with binary outcomes (success or failure); probabilities that are either documented or uncertain and updated from a Beta prior; a dynamic-programming (DP) accept-or-retry policy; Monte Carlo comparison with simpler player rules; a primary case study of documented Pokémon reset events in *Pokémon Scarlet* and *Violet*; and a smaller uncertain-odds test using published Path of Exile drop estimates.

Out of scope: branching story paths, RNG manipulation or reused seeds, live market bots, surveying players, and building an application. Code is a prototype used to compute policies and simulation results, not a product.

2. Computing and data science connection

Project type: hybrid. The *algorithmic* core is a finite-horizon dynamic program that returns an accept-or-retry action for each event, remaining time, and failure count. The *simulation* core is a Monte Carlo engine that compares that policy with simpler rules on identical random seeds. We are not building an app. Python with NumPy fills the DP table and runs the simulator; Matplotlib produces policy maps and confidence-interval plots.

2.1 Model

A play segment is a sequence of n events. Event i has success probability p_i per attempt, rewards $g_i > b_i$ for the good and bad outcomes, retry time c_i seconds, travel time f_i to the next event, and an entry cost e_i (zero when save-scumming; fragment cost when farming). After a failure the player keeps h_i: $h_i = 0$ when a reload discards the bad result, and $h_i = b_i - e_i$ when a failed farm still yields loot but the next run must be paid for.

Attempts are independent. The first attempt at each event costs no extra retry time but may carry e_i. After a failure the player either accepts b_i or retries. We assume one target outcome per event, a fixed event order, and that either outcome leads to the same later events. Rewards are treated as fixed within a session (player-stated values for Pokémon; a dated market snapshot for Path of Exile).

If the odds are documented we use $q_i(k) = p_i$. If they are uncertain, the player holds a Beta(α_i, β_i) prior and, after k failures, uses the posterior mean

$$q_i(k) = \frac{\alpha_i}{\alpha_i + \beta_i + k}.$$

A prior Beta$(1, 49)$ encodes a 2% belief; after 50 failed runs the estimate falls to 1%.

2.2 Main setting: fixed budget, solved by dynamic programming

After every decision the remainder of the problem is described by the current event i, time left t, and failures so far k. Let $V(i,t,k)$ be the best expected remaining reward, $W(i,t,k)$ the same just after a failure, and $N(i,t)$ the value of moving on:

$$N(i,t) = \begin{cases} V(i+1,\, t - f_i,\, 0) - e_{i+1} & \text{if } t \geq f_i \text{ and } i < n, \\ 0 & \text{otherwise,} \end{cases} \tag{1}$$

$$W(i,t,k) = \max\big\{b_i + N(i,t),\ h_i + V(i,\, t - c_i,\, k+1)\big\} \quad (t > c_i \text{ for retry}), \tag{2}$$

$$V(i,t,k) = q_i(k)\,(g_i + N(i,t)) + \big(1 - q_i(k)\big)\,W(i,t,k). \tag{3}$$

Moving on yields the next event's value minus its entry cost, or nothing if time runs out. A success is taken with probability $q_i(k)$. After a failure the player accepts b_i or retries, keeping h_i and returning to the same event with c_i less time and one more failure counted. The value of the session is $V(1, T, 0) - e_1$.

We fill the table backwards over events and upwards over t in whole seconds. With attempt cap K there are $n \times T \times K$ states and $O(nTK)$ time. For known odds ($K = 1$), ten events and a two-hour budget solve in under a second in Python. The output is a *policy map*: for each event and each remaining time, whether to retry or accept. We compute the full map because the best action can switch more than once as time runs down.

Toy instance. Two events, $T = 180$ seconds. Event 1: $p = 0.1$, $g = 10$, $b = 0$, $c = 20$. Event 2: $p = 0.5$, $g = 8$, $c = 10$. If event 2 is still ahead, the DP refuses a third retry on event 1; an "always reload" rule can spend the budget and miss event 2. That is the pattern Experiment D is designed to show on a larger scale.

2.3 Baseline: reward per second

With no deadline, time has an outside value ρ (reward per second available elsewhere). One more attempt at event i costs c_i seconds and gains $p_i(g_i - b_i) + h_i$ in expectation, so it is worth taking exactly when

$$p_i(g_i - b_i) + h_i \ \geq\ \rho\, c_i. \tag{4}$$

Every attempt at a given event has the same odds and cost, so the rule is all-or-nothing per event. We search for ρ so that the implied schedule is consistent with the value of time it assumes. With known odds and no deadline, a retry cap should not beat the better of "never retry" and "retry until success"; we check this numerically. With uncertain odds the claim need not hold, which hypothesis H5 tests.

2.4 Hypotheses and comparisons

The DP policy is optimal under the model's assumptions by construction. The empirical question is whether its advantage over simpler rules is large enough to matter. We test:

- **H1.** The DP's advantage over the best tuned baseline is largest at moderate time pressure and near zero for very short or very long budgets.
- **H3.** For an earlier event there is a smallest later reward at which the DP stops retrying it; we measure how

this threshold moves with T and c_i.

- **H5.** With uncertain odds, capping retries can beat retrying until success, unlike the known-odds case.

We also record whether the policy map switches more than once as time runs out, and how large an error in p_i or c_i must be before a simple rule matches the DP.

Table 1: Player strategies compared in simulation.

Strategy	**Rule after a bad roll at event i**
Never reload	Always accept.
Always reload	Reload whenever $t \geq c_i$.
Fixed-k	Up to k rolls per event ($k = 2, 3, 5$).
Reward-greedy	Reload only the top m events by $g_i - b_i$, until success, while $t \geq c_i + S_i$ (S_i: travel still needed to reach the last event).
Efficiency-greedy	Reload while $p_i(g_i - b_i)/c_i \geq \kappa$ and $t \geq c_i + S_i$, with cutoff κ tuned to its best value.
DP policy	The policy map from Section 2.2.

2.5 Simulation and analysis

Each simulated session has 5 to 10 events and budget T. A Python simulator plays at least 10,000 sessions per strategy on identical seeds, recording reward, reloads, and time spent reloading. We report means with 95% confidence intervals. Geometric waiting times in the simulator will be checked against $1/p$ so that the engine is not a black box.

Table 2: Planned experiments. Experiment D is the headline result.

Expt.	**What we vary**	**Question**
A. Budget	$T = 10, 20, 30, 60, 120$ min	When does the DP pull ahead of simple rules, and when does it not matter? (H1)
C. Retry cost	$c_i = 5, 20, 60, 120$ s	Whether cost alone flips the accept-or-retry decision.
D. Future opportunity	One event fixed; a high-value event added or removed later	Does a valuable later event make reloading the earlier one a bad decision? (H3)
B. Probability	p_i from 0.01 to 0.9	How unlikely must an event be before reloading it stops paying off?

Experiment D is the result we will lead with: a case where chasing an early reward is right on its own, but wrong because something better lies ahead.

3. Literature / background review

3.1 Save-scumming research

Save-scumming has received little academic attention. Farmer and Kelly [1] call for further study of the behaviour. Farmer, Kelly and Jones-Rincon [2, 3] examine why players reload, including avoiding frustration and exploring alternative outcomes. The 2026 preprint is recent, but the work remains psychological: it does not ask whether a reload is mathematically worthwhile when time is scarce and later events compete for the same minutes.

3.2 Related computational research

Restart research is close because both settings ask whether another attempt is worth its cost. Luby, Sinclair and Zuckerman [4] study restart policies for randomised algorithms; Lorenz [5] studies restarting under a deadline. Those models restart *one* randomised procedure, not a sequence of distinct later events with travel

times. Weitzman's costly-search model [6] compares an uncertain reward with the cost of continuing, but the decision maker chooses among alternatives rather than walking a prescribed route. Finite-state dynamic programming [10] supplies the computational method we use for the limited-playtime problem.

3.3 Bayesian learning and limited resources

For drops whose odds are not published, Bayesian decision theory updates a belief after each success or failure [7]. Russo et al. [8] describe the trade-off between exploration and exploitation: an attempt can yield both a reward and information. Bandits with Knapsacks [9] shows how learning changes when each attempt consumes a limited resource. That literature has a budget, but not a fixed event order with travel times between named encounters.

3.4 Research gap

Within the studies reviewed, we did not find work that allocates one real-time play budget across several sequential game events, with an accept-or-retry rule that can use a Beta belief when the odds are unknown. The project combines these established methods to compare an adaptive policy with simpler strategies such as fixed retry limits and reward-based rules.

4. Feasibility, limitations, and ethics

4.1 Data, methods, and workload

Almost all numerical evidence will come from our own simulator, whose true parameters we control. A working prototype of the DP and simulator already runs, and simulated means match the DP's predictions on small instances. Remaining work is experiment design, the case study, and analysis.

Primary case study (known odds). We model 4 to 6 resettable events in *Pokémon Scarlet/Violet* in a fixed order (for example a gift Pokémon, a static legendary, and a key battle). Encounter and shiny odds are taken from public documentation (Bulbapedia and published tables). Reset and travel times are averaged from at least three public gameplay videos per event. Any event for which documentation shows that a reset does not produce a fresh roll will be dropped. Nobody in the group needs to own the game.

Secondary test (uncertain odds). To contrast save-scumming with farming, we run the same DP on a small Path of Exile-style sequence in the simulator, using community drop-rate estimates and a *fixed* snapshot of market values (for example a dated poe.ninja extract). We will not scrape live markets, will not interact with game servers, and will not treat Path of Exile as a second full empirical study.

The remaining tasks are finalising the DP for both odds types, timing videos, taking one price snapshot, and writing the analysis. These divide cleanly among five members (Section 5).

4.2 Limitations

We will not claim beyond the following. Rewards in the Pokémon study are stated preferences, not facts; we test sensitivity rather than claiming a unique "true" value. Path of Exile prices will be a snapshot, not a live economy. Rolls are assumed independent; we do not model reused random seeds. Events follow a fixed order; branching paths are out of scope. Two illustrations show that the model can be instantiated on real published numbers. They do not show that it fits every genre.

4.3 Ethics

The project does not involve human subjects: there are no surveys, interviews, or recordings of players. We use published odds, community datasets, and public videos, and we record timestamps rather than republishing footage. We will not modify game clients or automate play on live servers, so terms-of-service issues do not arise. We take no position on whether save-scumming is good play; we evaluate a resource-allocation rule.

5. Timeline and team roles

The week-by-week plan is a starting point and may move, but every remaining milestone has a named lead. The contribution table records each member's primary role, key deliverables, and where that work appears in this plan or will appear in the repository.

Table 3: Remaining work, 6 October to 26 November 2026.

Week	Dates	Milestone	Lead
1	6 to 12 Oct	Freeze event list and parameters; DP solves the 2-event toy instance	Daniel
2	13 to 19 Oct	Simulator: 10k sessions for Never / Always / Fixed-k vs DP	Yera, Daniel, Roy
3	20 to 24 Oct	Interim progress meeting ($\sim$5 min); first policy-map figure	Yusif
4	27 Oct to 2 Nov	Video timings for 4 to 6 Pokémon events; odds table complete	Roy
5	3 to 4 Nov	Individual progress log; Experiments A and D; comparative analysis	Yera, Daniel, Roy
6	5 to 11 Nov	Uncertain-odds module and H5; ethics and limitations freeze	Daniel, Yera, Yusif
7	12 to 15 Nov	Final report due 15 Nov; figures, references, proofread	Yusif, Vivaan
8	17 to 26 Nov	10-minute group presentation (slot TBA)	all

Table 4: Team contributions by member.

Member	Primary role	Key deliverables	Plan / repository evidence
Vu Q. Nguyen *(Daniel)*	Algorithm lead	DP states (i, t, k); policy map; uncertain-odds (Beta) module; experiment support with Yera	Model and DP sections; forthcoming DP and policy-map code
Yerassyl Taisarinov *(Yera)*	Simulation lead	Monte Carlo engine; shared seeds; 95% CIs; experiment figures and tables	Simulation and experiment sections; forthcoming simulator and plots
Jihun Choi *(Roy)*	Experiments and case study	Experiment design; comparative analysis; Pokémon event list, odds table, and video timings	Experiment table; case-study plan; forthcoming odds and timing notes
Yusif Seyidov *(Yusif)*	Coordination and report	Team forming and meetings; milestone tracking; teacher consults; plan-scope decisions; feasibility, ethics, and limitations; quality check and edits for Sections 1–4	Ethics and limitations; this contribution table; report assembly and presentation script
Vivaan Kandhari *(Vivaan)*	Literature and baseline	Literature review and research gap; ρ-rule baseline; Path of Exile snapshot	Literature / gap section; baseline section; forthcoming dated price extract

References

[1] G. Farmer and D. Kelly, " 'Save-scumming' in role-playing games: A call for further psychological exploration," British Psychological Society, Mar. 2024. [Online]. Available: https://www.bps.org.uk/news

[2] G. Farmer, D. Kelly and A. Jones-Rincon, "Save-Scumming: A Novel Exploration of Video Game Play," in *Abstract Proceedings of DiGRA 2025: Games at the Crossroads*, 2025, doi: 10.26503/dl.v2025i3.2509.

[3] G. Farmer, D. Kelly and A. Jones-Rincon, " 'You're playing the game wrong': Exploring motivations and perceptions of save-scumming in video games," PsyArXiv preprint, Jun. 2026, doi: 10.31234/osf.io/ew7nj_v2.

[4] M. Luby, A. Sinclair and D. Zuckerman, "Optimal speedup of Las Vegas algorithms," *Information Processing Letters*, vol. 47, no. 4, pp. 173-180, 1993, doi: 10.1016/0020-0190(93)90029-9.

[5] J.-H. Lorenz, "Completion probabilities and parallel restart strategies under an imposed deadline," *PLOS ONE*, vol. 11, no. 10, Art. no. e0164605, 2016, doi: 10.1371/journal.pone.0164605.

[6] M. L. Weitzman, "Optimal search for the best alternative," *Econometrica*, vol. 47, no. 3, pp. 641-654, 1979, doi: 10.2307/1910412.

[7] M. H. DeGroot, *Optimal Statistical Decisions*. Hoboken, NJ: John Wiley & Sons, 2004, doi: 10.1002/0471729000.

[8] D. J. Russo, B. Van Roy, A. Kazerouni, I. Osband and Z. Wen, "A tutorial on Thompson sampling," *Foundations and Trends in Machine Learning*, vol. 11, no. 1, pp. 1-96, 2018, doi: 10.1561/2200000070.

[9] A. Badanidiyuru, R. Kleinberg and A. Slivkins, "Bandits with knapsacks," *Journal of the ACM*, vol. 65, no. 3, Art. no. 13, pp. 1-55, 2018, doi: 10.1145/3164539.

[10] T. J. Sargent and J. Stachurski, *Dynamic Programming: Finite States*. Cambridge: Cambridge University Press, 2025, doi: 10.1017/9781009540780.